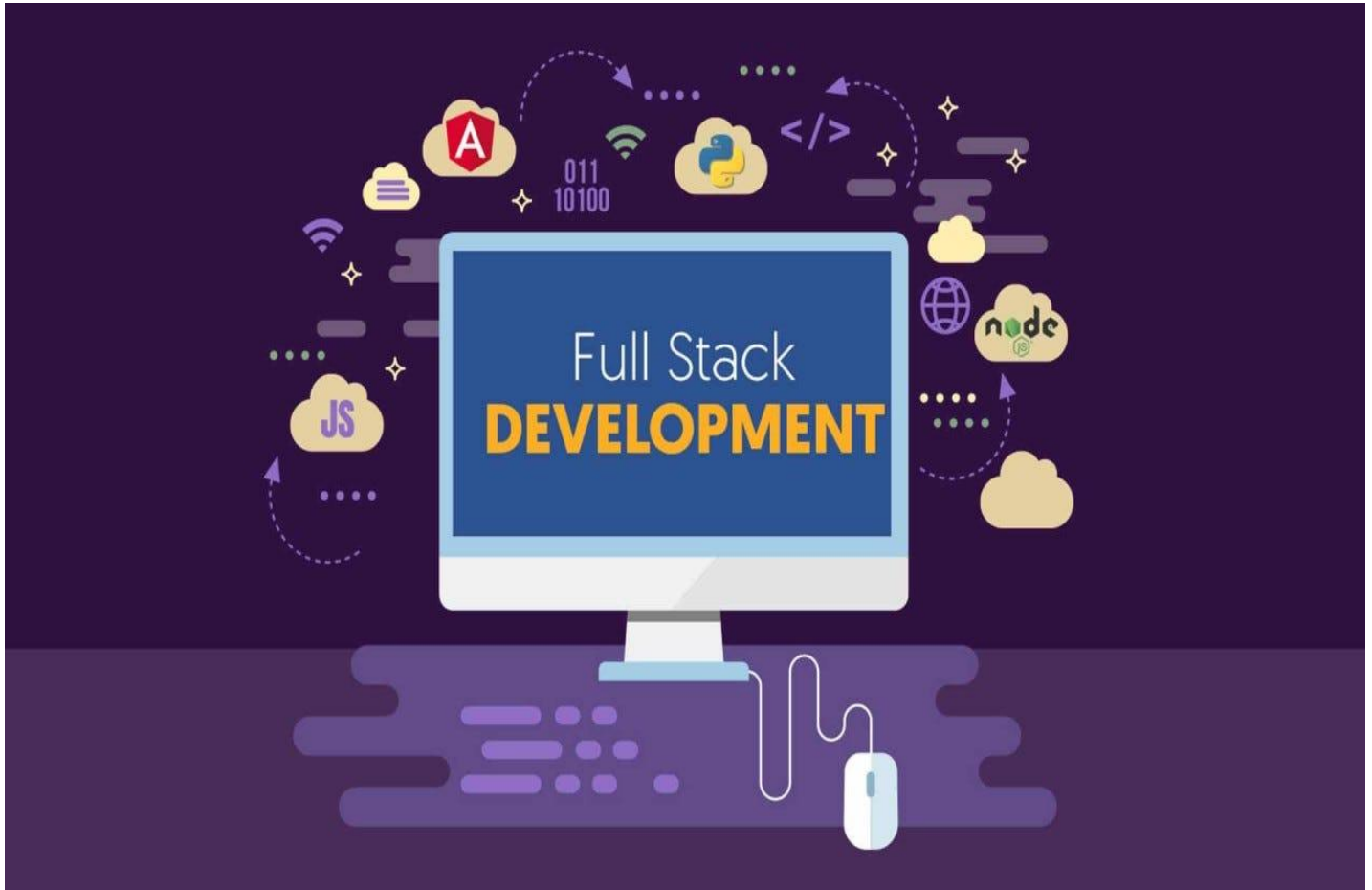




Institute of  
**Computer Science &  
Application**

DEPARTMENT OF COMPUTER APPLICATION

# Full Stack Development

(SSCA4020) | Institute of Computer Science & Applications

Name of Student: Shah Rishabh Alpeshbhai

Enrolment No.: 23SS02IT181

Academic Year: 2026-27



**P P SAVANI UNIVERSITY**  
**INSTITUTE OF COMPUTER SCIENCE & APPLICATION**

**CERTIFICATE**

This is to certify that

Mr./Ms. **Shah Rishabh Alpeshbhai**

of **B.Sc. (Hons) IT - Sem 7** Program

having Enrollment No. **23SS02IT181** has completed

his/her Term work in the subject of

**FULL STACK DEVELOPMENT (SSCA4020).**

Marks Obtained: \_\_\_\_\_ out of \_\_\_\_\_

Sign of Faculty

Date: \_\_\_\_\_

Sign of Head of Department

Date: \_\_\_\_\_

# INDEX

| Sr. | Title of Practical                                                                                                                                                                          | Page | Date | Marks | Sign |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------|-------|------|
| 01  | Install VS Code, Git, and configure the development and create GitHub account and push your first repository                                                                                | 4    |      |       |      |
| 02  | Create a personal portfolio webpage using HTML5 semantic tags and Apply CSS3 styles (Flexbox, Grid, media queries) for responsiveness                                                       | 6    |      |       |      |
| 03  | Browser interaction and form validations (Web browser environments, forms and validations, image sliders) [Image slider plugins of jQuery, Client- side validation of Registration & Login. | 11   |      |       |      |
| 04  | Design a responsive landing page using Tailwind. Implement a Tailwind navigation bar for a product page.platforms                                                                           | 15   |      |       |      |
| 05  | Make interactive web pages with reactJS concepts                                                                                                                                            | 18   |      |       |      |
| 06  | Design web application with back end of NodeJS,                                                                                                                                             | 21   |      |       |      |
| 07  | Connect Node.js with MongoDB and Perform CRUD operations (Create, Read, Update, Delete) on a MongoDB collection. Integrate MySQL database and execute SQL queries from Node.js.             | 24   |      |       |      |
| 08  | Develop Complete Web application as a mini project.                                                                                                                                         | 27   |      |       |      |

## PRACTICAL 1

To install Visual Studio Code and Git, configure the development environment, create a GitHub account and push the first repository.

### STEPS:

1. Download and install Visual Studio Code from [code.visualstudio.com](https://code.visualstudio.com) and Git from [git-scm.com](https://git-scm.com) (keep the default options).
2. Create a free GitHub account at [github.com](https://github.com) and create a new empty repository named my-first-repo.
3. Open the VS Code integrated terminal (Ctrl + `) and configure Git with the user name and e-mail.
4. Create a local repository, add the project files, commit them and push to GitHub as shown below.

### CODE:

#### Git commands (terminal)

```
# Check installation and configure Git
git --version
git config --global user.name "Rishabh Shah"
git config --global user.email "rishabh.shah@example.com"
git config --global --list

# Create the local repository
mkdir my-first-repo
cd my-first-repo
git init -b main

# Add files, commit and view history
git add .
git commit -m "Initial commit: add README and index.html"
git log --oneline

# Connect to GitHub and push
git remote add origin https://github.com/<your-username>/my-first-repo.git
git push -u origin main
```

#### README.md

```
# My First Repository
Created by Shah Rishabh Alpeshbhai (23SS02IT181) for Full Stack Development Lab.
```

#### index.html

```
<!DOCTYPE html>
<html><head><title>Hello Git</title></head>
<body><h1>Hello, GitHub!</h1></body></html>
```

## OUTPUT:

```
Windows PowerShell

PS C:\Users\Rishabh\my-first-repo> git --version
git version 2.43.0.windows.1
PS C:\Users\Rishabh\my-first-repo> git config --global user.name "Rishabh Shah"
PS C:\Users\Rishabh\my-first-repo> git config --global user.email "rishabh.shah@example.com"
PS C:\Users\Rishabh\my-first-repo> git config --global --list
user.name=Rishabh Shah
user.email=rishabh.shah@example.com
PS C:\Users\Rishabh\my-first-repo> mkdir my-first-repo; cd my-first-repo
PS C:\Users\Rishabh\my-first-repo> git init -b main
Initialized empty Git repository in C:/Users/Rishabh/my-first-repo/.git/
PS C:\Users\Rishabh\my-first-repo> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    index.html

nothing added to commit but untracked files present (use "git add" to track)
```

```
Windows PowerShell

PS C:\Users\Rishabh\my-first-repo> git add .
PS C:\Users\Rishabh\my-first-repo> git commit -m "Initial commit: add README and index.html"
[main (root-commit) c94d257] Initial commit: add README and index.html
 2 files changed, 5 insertions(+)
 create mode 100644 README.md
 create mode 100644 index.html
PS C:\Users\Rishabh\my-first-repo> git log --oneline
c94d257 (HEAD -> main) Initial commit: add README and index.html
PS C:\Users\Rishabh\my-first-repo> git remote add origin https://github.com/<your-username>/my-first-repo.git
PS C:\Users\Rishabh\my-first-repo> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 440 bytes | 440.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/<your-username>/my-first-repo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

## PRACTICAL 2

To create a personal portfolio webpage using HTML5 semantic tags and apply CSS3 styles (Flexbox, Grid, media queries) for responsiveness.

### CODE:

#### index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Rishabh Shah | Portfolio</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <h1 class="logo">Rishabh<span></span></h1>
    <nav>
      <ul class="nav-links">
        <li><a href="#about">About</a></li>
        <li><a href="#skills">Skills</a></li>
        <li><a href="#projects">Projects</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section class="hero" id="home">
      <div class="hero-text">
        <p class="hello">Hello, I am</p>
        <h2>Shah Rishabh Alpeshbhai</h2>
        <p class="role">B.Sc. (Hons) Information Technology Student</p>
        <a href="#contact" class="btn">Contact Me</a>
      </div>
      <figure class="avatar">RS</figure>
    </section>

    <section id="about">
      <h2 class="section-title">About Me</h2>
      <article>
        <p>I am a 7th semester student of B.Sc. (Hons) Information Technology at P P Savani University (Enrollment No. 23SS02IT181). I enjoy building responsive websites and learning full stack development with HTML, CSS, JavaScript, React, Node.js and databases.</p>
      </article>
    </section>

    <section id="skills">
      <h2 class="section-title">Skills</h2>
      <div class="skills-grid">
        <div class="skill">HTML5</div>
        <div class="skill">CSS3</div>
        <div class="skill">JavaScript</div>
        <div class="skill">React</div>
        <div class="skill">Node.js</div>
        <div class="skill">MongoDB</div>
        <div class="skill">MySQL</div>
        <div class="skill">Git & GitHub</div>
      </div>
    </section>

    <section id="projects">
      <h2 class="section-title">Projects</h2>
      <div class="cards">
        <article class="card">
          <h3>Portfolio Website</h3>
```

```

    <p>Responsive personal website built using semantic HTML5, Flexbox and CSS Grid.</p>
  </article>
  <article class="card">
    <h3>Login & Registration</h3>
    <p>Client side form validation with jQuery and an image slider.</p>
  </article>
  <article class="card">
    <h3>Task Manager</h3>
    <p>Full stack web application using Node.js, Express and MongoDB.</p>
  </article>
</div>
</section>

<aside class="quote">
  <p>"First, solve the problem. Then, write the code."</p>
</aside>

<section id="contact">
  <h2 class="section-title">Contact</h2>
  <address>
    Email: rishabh.shah@example.com<br>
    Surat, Gujarat, India
  </address>
</section>
</main>

<footer>
  <p>&copy; 2026 Shah Rishabh Alpeshbhai | 23SS02IT181</p>
</footer>
</body>
</html>

```

### style.css

```

* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: 'Segoe UI', Arial, sans-serif; line-height: 1.6; color: #2d3142; background: #f7f8fc; }

/* ----- Header (Flexbox) ----- */
.site-header { display: flex; justify-content: space-between; align-items: center;
padding: 18px 8%; background: #1f2a44; color: #fff; position: sticky; top: 0; }
.logo span { color: #ff7a59; }
.nav-links { display: flex; gap: 28px; list-style: none; }
.nav-links a { color: #fff; text-decoration: none; font-weight: 500; }
.nav-links a:hover { color: #ff7a59; }

/* ----- Hero (Flexbox) ----- */
.hero { display: flex; align-items: center; justify-content: space-between;
gap: 40px; padding: 80px 8%; background: linear-gradient(135deg, #1f2a44, #3d5a80); color: #fff; }
.hello { color: #ff7a59; font-weight: 600; }
.hero h2 { font-size: 2.6rem; margin: 6px 0; }
.role { margin-bottom: 24px; opacity: .9; }
.btn { display: inline-block; padding: 12px 28px; background: #ff7a59; color: #fff;
border-radius: 30px; text-decoration: none; font-weight: 600; }
.avatar { width: 200px; height: 200px; border-radius: 50%; background: #ff7a59;
display: flex; align-items: center; justify-content: center; font-size: 4rem; font-weight: 700;
border: 6px solid rgba(255,255,255,.4); }

section { padding: 60px 8%; }
.section-title { text-align: center; margin-bottom: 30px; font-size: 2rem; color: #1f2a44; }
#about article { max-width: 800px; margin: auto; text-align: center; }

/* ----- Skills (CSS Grid) ----- */
.skills-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 20px; }
.skill { background: #fff; padding: 22px; text-align: center; border-radius: 12px;
font-weight: 600; box-shadow: 0 4px 14px rgba(0,0,0,.08); }

/* ----- Projects (CSS Grid) ----- */
.cards { display: grid; grid-template-columns: repeat(3, 1fr); gap: 24px; }
.card { background: #fff; padding: 26px; border-radius: 12px; border-top: 5px solid #ff7a59;
box-shadow: 0 4px 14px rgba(0,0,0,.08); }
.card h3 { margin-bottom: 8px; color: #1f2a44; }

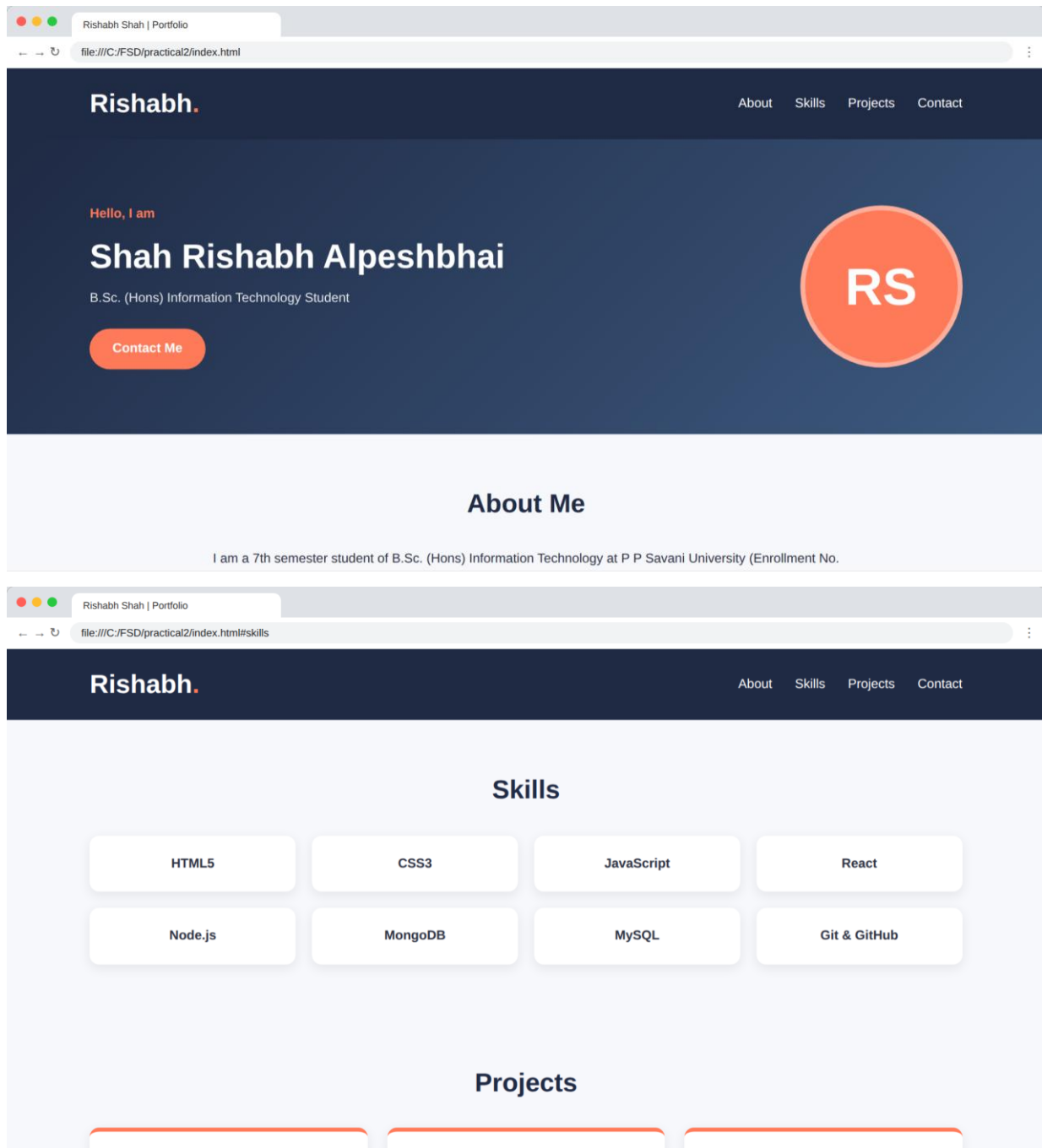
.quote { background: #ffe8e0; padding: 30px 8%; text-align: center; font-style: italic; font-size: 1.2rem; }

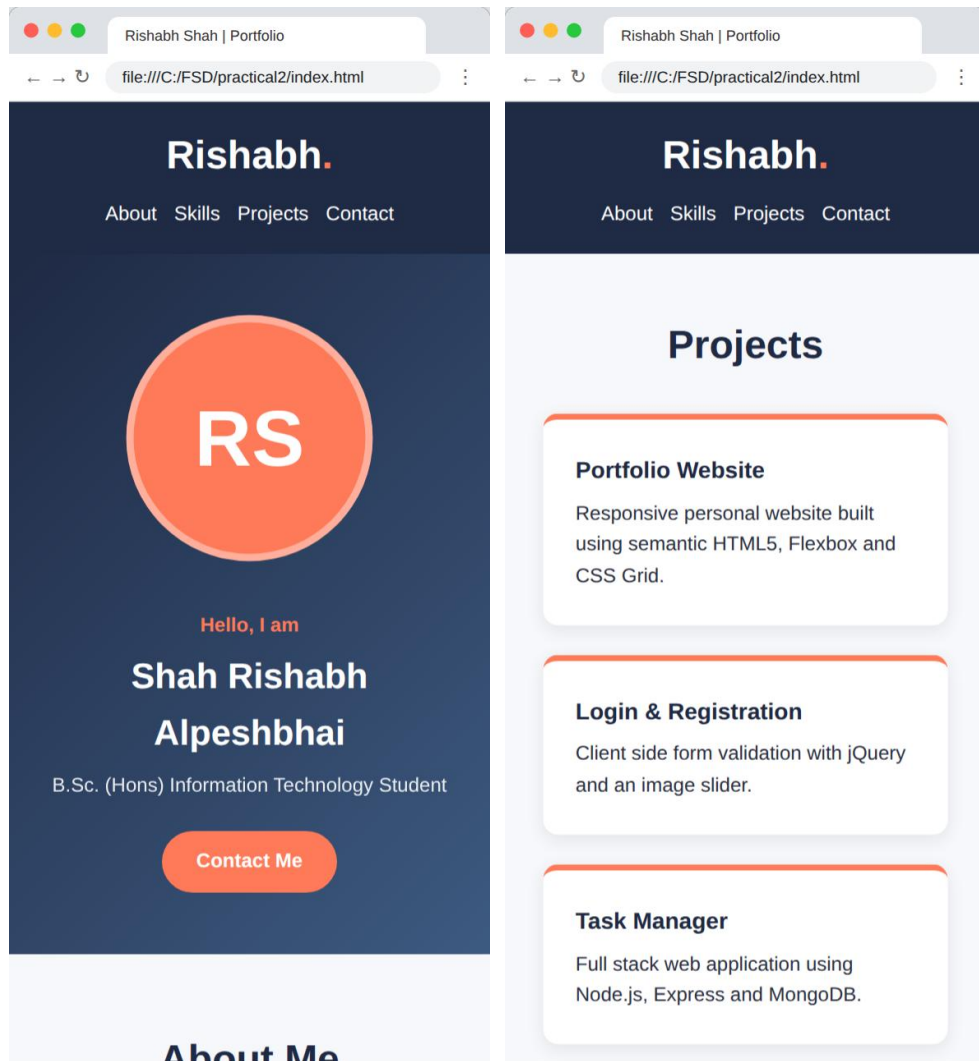
```

```
#contact { text-align: center; }
address { font-style: normal; font-size: 1.1rem; }
footer { background: #1f2a44; color: #fff; text-align: center; padding: 18px; }

/* ----- Media Queries (Responsive) ----- */
@media (max-width: 900px) {
  .skills-grid { grid-template-columns: repeat(2, 1fr); }
  .cards { grid-template-columns: 1fr 1fr; }
}
@media (max-width: 600px) {
  .site-header { flex-direction: column; gap: 10px; }
  .nav-links { gap: 14px; }
  .hero { flex-direction: column-reverse; text-align: center; padding: 50px 6%; }
  .hero h2 { font-size: 1.8rem; }
  .cards { grid-template-columns: 1fr; }
}
```

## OUTPUT:





## PRACTICAL 3

To implement browser interaction and form validations: an image slider using the jQuery Slick plugin and client-side validation of Registration and Login forms.

**AIM:** To implement browser interaction and form validations: an image slider using the jQuery Slick plugin and client-side validation of Registration and Login forms.

### CODE:

#### index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Registration, Login and Slider</title>
  <link rel="stylesheet" href="lib/slick.css">
  <link rel="stylesheet" href="lib/slick-theme.css">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 class="title">Practical 3 - Form Validation & Image Slider</h1>

  <!-- jQuery Image Slider (Slick plugin) -->
  <div class="slider">
    <div></div>
    <div></div>
    <div></div>
  </div>

  <div class="forms">
    <!-- Registration Form -->
    <form id="regForm" novalidate>
      <h2>Registration</h2>
      <label>Full Name</label>
      <input type="text" id="name" placeholder="Enter full name">
      <span class="error" id="nameErr"></span>

      <label>Email</label>
      <input type="text" id="email" placeholder="Enter email">
      <span class="error" id="emailErr"></span>

      <label>Mobile Number</label>
      <input type="text" id="mobile" placeholder="10 digit mobile number">
      <span class="error" id="mobileErr"></span>

      <label>Password</label>
      <input type="password" id="password" placeholder="Minimum 6 characters">
      <span class="error" id="passErr"></span>

      <label>Confirm Password</label>
      <input type="password" id="cpassword" placeholder="Re-enter password">
      <span class="error" id="cpassErr"></span>

      <button type="submit">Register</button>
      <p class="success" id="regMsg"></p>
    </form>

    <!-- Login Form -->
    <form id="loginForm" novalidate>
      <h2>Login</h2>
      <label>Email</label>
      <input type="text" id="lemail" placeholder="Enter email">
      <span class="error" id="lemailErr"></span>

      <label>Password</label>
      <input type="password" id="lpassword" placeholder="Enter password">
      <span class="error" id="lpassErr"></span>
```

```

        <button type="submit">Login</button>
        <p class="success" id="loginMsg"></p>
    </form>
</div>

<script src="lib/jquery.min.js"></script>
<script src="lib/slick.min.js"></script>
<script src="script.js"></script>
</body>
</html>

```

### style.css

```

body { font-family: Arial, sans-serif; background: #eef1f7; margin: 0; padding: 20px; }
.title { text-align: center; color: #1f2a44; }
.slider { width: 700px; margin: 20px auto 40px; }
.slider img { width: 100%; border-radius: 10px; display: block; }
.forms { display: flex; justify-content: center; gap: 40px; flex-wrap: wrap; }
form { background: #fff; padding: 25px 30px; width: 320px; border-radius: 10px;
    box-shadow: 0 4px 14px rgba(0,0,0,.12); }
form h2 { text-align: center; margin-top: 0; color: #1f2a44; }
label { display: block; margin-top: 12px; font-weight: bold; font-size: 14px; }
input { width: 100%; padding: 9px; margin-top: 5px; border: 1px solid #aaa;
    border-radius: 5px; box-sizing: border-box; }
input.invalid { border-color: #d93025; }
input.valid { border-color: #1e8e3e; }
.error { color: #d93025; font-size: 12px; display: block; min-height: 14px; }
.success { color: #1e8e3e; font-weight: bold; text-align: center; }
button { width: 100%; margin-top: 15px; padding: 10px; background: #1f2a44; color: #fff;
    border: none; border-radius: 5px; font-size: 16px; cursor: pointer; }
button:hover { background: #ff7a59; }

```

### script.js

```

$(document).ready(function () {

    // ----- jQuery Image Slider (Slick plugin) -----
    $('.slider').slick({
        dots: true,
        arrows: true,
        autoplay: true,
        autoplaySpeed: 2500,
        infinite: true
    });

    // ----- Helper -----
    function setStatus(input, errBox, msg) {
        if (msg) {
            input.addClass('invalid').removeClass('valid');
            errBox.text(msg);
            return false;
        }
        input.addClass('valid').removeClass('invalid');
        errBox.text("");
        return true;
    }

    var emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    var mobileRegex = /^[6-9][0-9]{9}$/;

    // ----- Registration validation -----
    $('#regForm').on('submit', function (e) {
        e.preventDefault();
        var name = $('#name').val().trim();
        var email = $('#email').val().trim();
        var mobile = $('#mobile').val().trim();
        var pass = $('#password').val();
        var cpass = $('#cpassword').val();

        var ok = true;
        ok = setStatus($('#name'), $('#nameErr'),
            name.length < 3 ? 'Name must be at least 3 characters' : '') && ok;
        ok = setStatus($('#email'), $('#emailErr'),
            !emailRegex.test(email) ? 'Enter a valid email address' : '') && ok;
    });

```

```

ok = setStatus($('#mobile'), $('#mobileErr'),
    !mobileRegex.test(mobile) ? 'Enter valid 10 digit mobile number' : '') && ok;
ok = setStatus($('#password'), $('#passErr'),
    pass.length < 6 ? 'Password must be at least 6 characters' : '') && ok;
ok = setStatus($('#cpassword'), $('#cpassErr'),
    (cpass !== pass || cpass === '') ? 'Passwords do not match' : '') && ok;

$('#regMsg').text(ok ? 'Registration successful!' : '');
});

// ----- Login validation -----
$('#loginForm').on('submit', function (e) {
    e.preventDefault();
    var email = $('#lemail').val().trim();
    var pass = $('#lpassword').val();
    var ok = true;
    ok = setStatus($('#lemail'), $('#lemailErr'),
        !emailRegex.test(email) ? 'Enter a valid email address' : '') && ok;
    ok = setStatus($('#lpassword'), $('#lpassErr'),
        pass.length < 6 ? 'Password must be at least 6 characters' : '') && ok;
    $('#loginMsg').text(ok ? 'Login successful!' : '');
});
});

```

## OUTPUT:

The screenshot shows a web browser window with the title "Registration, Login and Slider". The address bar shows the file path "file:///C:/FSD/practical3/index.html". The main content area has a light blue background. At the top, there's a dark blue banner with the text "Welcome to Full Stack Development" and a logo consisting of five overlapping circles. Below the banner, there are two white boxes with rounded corners. The left box is titled "Registration" and contains five input fields: "Full Name" (placeholder: "Enter full name"), "Email" (placeholder: "Enter email"), "Mobile Number" (placeholder: "10 digit mobile number"), "Password" (placeholder: "Minimum 6 characters"), and "Confirm Password" (placeholder: "Re-enter password"). A dark blue "Register" button is at the bottom. The right box is titled "Login" and contains two input fields: "Email" (placeholder: "Enter email") and "Password" (placeholder: "Enter password"). A dark blue "Login" button is at the bottom.

Registration, Login and Slider

file:///C:/FSD/practical3/index.html

### Practical 3 - Form Validation & Image Slider

Welcome to Full Stack Development

#### Registration

**Full Name**

Name must be at least 3 characters

**Email**

Enter a valid email address

**Mobile Number**

Enter valid 10 digit mobile number

**Password**

Password must be at least 6 characters

**Confirm Password**

Passwords do not match

Register

#### Login

**Email**

Enter a valid email address

**Password**

Password must be at least 6 characters

Login

Registration, Login and Slider

file:///C:/FSD/practical3/index.html

### Practical 3 - Form Validation & Image Slider

Welcome to Full Stack Development

#### Registration

**Full Name**

**Email**

**Mobile Number**

**Password**

**Confirm Password**

Register

Registration successful!

#### Login

**Email**

**Password**

Login

Login successful!

## PRACTICAL 4

: To design a responsive landing page using Tailwind CSS and implement a Tailwind navigation bar for a product page.

**AIM:** To design a responsive landing page using Tailwind CSS and implement a Tailwind navigation bar for a product page.

### STEPS:

1. Install Tailwind CSS with npm, create the configuration file and the input CSS file.
2. Write the HTML using Tailwind utility classes and build the CSS file with the Tailwind CLI.

### CODE:

#### Commands

```
npm init -y
npm install -D tailwindcss@3
npx tailwindcss init
npx tailwindcss -i ./input.css -o ./output.css --watch
```

#### tailwind.config.js

```
module.exports = {
  content: ['./index.html'],
  theme: { extend: {} },
  plugins: [],
}
```

#### input.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

#### index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>NovaPods - Product Landing Page</title>
  <link href="output.css" rel="stylesheet">
</head>
<body class="bg-gray-50 text-gray-800">

  <!-- ===== Tailwind Navigation Bar ===== -->
  <nav class="bg-white shadow sticky top-0 z-50">
    <div class="max-w-6xl mx-auto px-4 flex items-center justify-between h-16">
      <a href="#" class="text-2xl font-extrabold text-indigo-600">Nova<span class="text-pink-500">Pods</span></a>

      <ul class="hidden md:flex items-center space-x-8 font-medium">
        <li><a href="#home" class="hover:text-indigo-600">Home</a></li>
        <li><a href="#features" class="hover:text-indigo-600">Features</a></li>
        <li><a href="#pricing" class="hover:text-indigo-600">Pricing</a></li>
        <li><a href="#contact" class="hover:text-indigo-600">Contact</a></li>
      </ul>

      <a href="#pricing" class="hidden md:inline-block bg-indigo-600 text-white px-5 py-2 rounded-full font-semibold hover:bg-indigo-700">Buy
Now</a>

      <!-- mobile menu button -->
      <button class="md:hidden text-3xl text-indigo-600" aria-label="menu">&#9776;</button>
    </div>
  </nav>

  <!-- ===== Hero Section ===== -->
  <section id="home" class="bg-gradient-to-r from-indigo-600 to-purple-600 text-white">
    <div class="max-w-6xl mx-auto px-4 py-20 grid md:grid-cols-2 gap-10 items-center">
      <div>
```

```

<p class="uppercase tracking-widest text-pink-200 font-semibold">New Launch 2026</p>
<h1 class="text-4xl md:text-5xl font-extrabold mt-3 leading-tight">Wireless Earbuds That Move With You</h1>
<p class="mt-5 text-indigo-100 text-lg">Crystal clear sound, active noise cancellation and 40 hours of battery life.</p>
<div class="mt-8 flex gap-4">
  <a href="#pricing" class="bg-white text-indigo-700 px-6 py-3 rounded-lg font-bold shadow hover:bg-gray-100">Shop Now</a>
  <a href="#features" class="border-2 border-white px-6 py-3 rounded-lg font-bold hover:bg-white hover:text-indigo-700">Learn More</a>
</div>
</div>
<div class="flex justify-center">
  <div class="w-64 h-64 md:w-80 md:h-80 rounded-full bg-white/20 flex items-center justify-center text-8xl shadow-2xl">#127911</div>
</div>
</div>
</section>

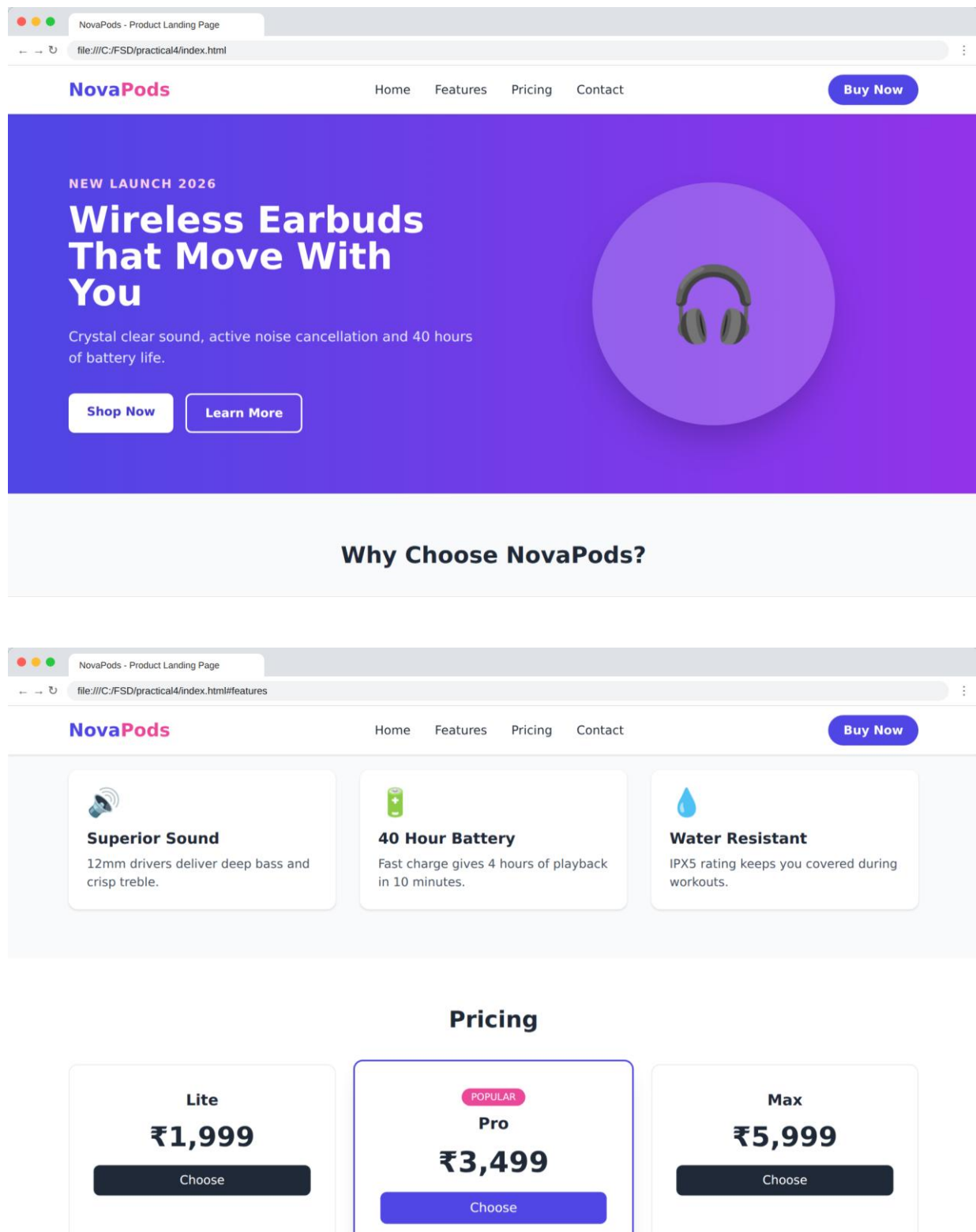
<!-- ===== Features ===== -->
<section id="features" class="max-w-6xl mx-auto px-4 py-16">
  <h2 class="text-3xl font-bold text-center mb-10">Why Choose NovaPods?</h2>
  <div class="grid sm:grid-cols-2 lg:grid-cols-3 gap-8">
    <div class="bg-white p-6 rounded-xl shadow hover:shadow-xl transition">
      <div class="text-4xl mb-3">#128266</div>
      <h3 class="text-xl font-semibold mb-2">Superior Sound</h3>
      <p class="text-gray-600">12mm drivers deliver deep bass and crisp treble.</p>
    </div>
    <div class="bg-white p-6 rounded-xl shadow hover:shadow-xl transition">
      <div class="text-4xl mb-3">#128267</div>
      <h3 class="text-xl font-semibold mb-2">40 Hour Battery</h3>
      <p class="text-gray-600">Fast charge gives 4 hours of playback in 10 minutes.</p>
    </div>
    <div class="bg-white p-6 rounded-xl shadow hover:shadow-xl transition">
      <div class="text-4xl mb-3">#128167</div>
      <h3 class="text-xl font-semibold mb-2">Water Resistant</h3>
      <p class="text-gray-600">IPX5 rating keeps you covered during workouts.</p>
    </div>
  </div>
</section>

<!-- ===== Pricing ===== -->
<section id="pricing" class="bg-white py-16">
  <div class="max-w-6xl mx-auto px-4">
    <h2 class="text-3xl font-bold text-center mb-10">Pricing</h2>
    <div class="grid md:grid-cols-3 gap-8">
      <div class="border rounded-xl p-8 text-center">
        <h3 class="text-xl font-semibold">Lite</h3>
        <p class="text-4xl font-extrabold my-4">#8377;1,999</p>
        <button class="bg-gray-800 text-white w-full py-2 rounded-lg hover:bg-gray-900">Choose</button>
      </div>
      <div class="border-2 border-indigo-600 rounded-xl p-8 text-center shadow-xl scale-105">
        <span class="bg-pink-500 text-white text-xs px-3 py-1 rounded-full">POPULAR</span>
        <h3 class="text-xl font-semibold mt-2">Pro</h3>
        <p class="text-4xl font-extrabold my-4">#8377;3,499</p>
        <button class="bg-indigo-600 text-white w-full py-2 rounded-lg hover:bg-indigo-700">Choose</button>
      </div>
      <div class="border rounded-xl p-8 text-center">
        <h3 class="text-xl font-semibold">Max</h3>
        <p class="text-4xl font-extrabold my-4">#8377;5,999</p>
        <button class="bg-gray-800 text-white w-full py-2 rounded-lg hover:bg-gray-900">Choose</button>
      </div>
    </div>
  </div>
</section>

<footer id="contact" class="bg-gray-900 text-gray-300 text-center py-6">
  &copy; 2026 NovaPods | Shah Rishabh Alpeshbhai (23SS02IT181)
</footer>
</body>
</html>

```

## OUTPUT:



## PRACTICAL 5

To make interactive web pages using ReactJS concepts: components, props, state (useState), events, controlled inputs, list rendering and useEffect.

### STEPS:

1. Install React, ReactDOM and a bundler (esbuild).
2. Create the components in src/index.jsx, style them with app.css and build the bundle.

### CODE:

#### Commands

```
npm init -y
npm install react react-dom esbuild
npx esbuild src/index.jsx --bundle --outdir=dist --minify
```

#### src/index.jsx

```
import React, { useState, useEffect } from "react";
import { createRoot } from "react-dom/client";
import "./app.css";

/* ----- Component with props ----- */
function Header({ title, student }) {
  return (
    <header className="hdr">
      <h1>{title}</h1>
      <p>{student}</p>
    </header>
  );
}

/* ----- useState : Counter ----- */
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div className="card">
      <h2>Counter</h2>
      <div className="count">{count}</div>
      <button onClick={() => setCount(count + 1)}>+ Increment</button>
      <button onClick={() => setCount(count - 1)}>- Decrement</button>
      <button className="gray" onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

/* ----- Controlled input + live preview ----- */
function Greeting() {
  const [name, setName] = useState("");
  return (
    <div className="card">
      <h2>Controlled Input</h2>
      <input
        placeholder="Type your name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <p className="hello">{name ? `Hello, ${name}! Welcome to React.` : "Please enter your name"}</p>
    </div>
  );
}

/* ----- List rendering, events, conditional rendering ----- */
function TodoApp() {
  const [text, setText] = useState("");
  const [todos, setTodos] = useState([
    { id: 1, title: "Learn JSX", done: true },
    { id: 2, title: "Understand useState and useEffect", done: false },
  ]);
}
```

```

const addTodo = () => {
  if (text.trim() === "") return;
  setTodos([...todos, { id: Date.now(), title: text, done: false }]);
  setText("");
};
const toggle = (id) =>
  setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
const remove = (id) => setTodos(todos.filter((t) => t.id !== id));

// useEffect : runs whenever todos change
useEffect(() => {
  document.title = `Todos (${todos.filter((t) => !t.done).length} pending)`;
}, [todos]);

return (
  <div className="card wide">
    <h2>Todo List</h2>
    <div className="row">
      <input
        placeholder="Add a new task"
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <button onClick={addTodo}>Add</button>
    </div>
    <ul>
      {todos.map((t) => (
        <li key={t.id}>
          <span className={t.done ? "done" : ""} onClick={() => toggle(t.id)}>
            {t.done ? "✓" : "○"} {t.title}
          </span>
          <button className="del" onClick={() => remove(t.id)}>Delete</button>
        </li>
      ))}
    </ul>
    {todos.length === 0 && <p>No tasks left!</p>}
    <p className="small">Pending: {todos.filter((t) => !t.done).length}</p>
  </div>
);
}

function App() {
  return (
    <div>
      <Header title="React Interactive Web Page" student="Shah Rishabh Alpeshbhai | 23SS02IT181" />
      <div className="grid">
        <Counter />
        <Greeting />
        <TodoApp />
      </div>
    </div>
  );
}

createRoot(document.getElementById("root")).render(<App />);

src/app.css
body { margin: 0; font-family: Arial, sans-serif; background: #f0f4ff; }
.hdr { background: #20232a; color: #61dafb; text-align: center; padding: 18px; }
.hdr p { color: #ddd; margin: 4px 0 0; }
.grid { display: flex; flex-wrap: wrap; gap: 24px; justify-content: center; padding: 30px; }
.card { background: #fff; width: 280px; padding: 20px; border-radius: 12px; box-shadow: 0 4px 12px rgba(0,0,0,.12); text-align: center; }
.card.wide { width: 440px; text-align: left; }
.card h2 { margin-top: 0; color: #20232a; }
.count { font-size: 56px; font-weight: bold; color: #0b7285; margin: 10px 0; }
button { background: #0b7285; color: #fff; border: 0; padding: 8px 14px; border-radius: 6px; cursor: pointer; margin: 3px; }
button.gray { background: #868e96; } button.del { background: #e03131; padding: 4px 10px; font-size: 12px; }
input { padding: 8px; border: 1px solid #999; border-radius: 6px; width: 70%; }
.row { display: flex; gap: 6px; } .row input { flex: 1; }
.hello { color: #2b8a3e; font-weight: bold; }
ul { list-style: none; padding: 0; } li { display: flex; justify-content: space-between; align-items: center; padding: 8px 0; border-bottom: 1px solid #eee; }

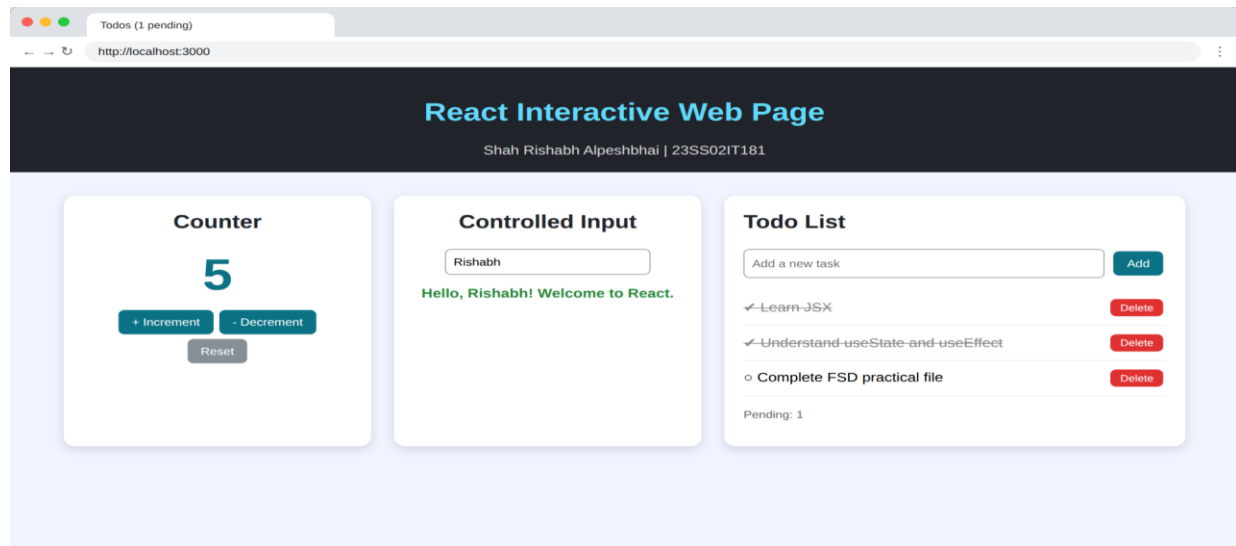
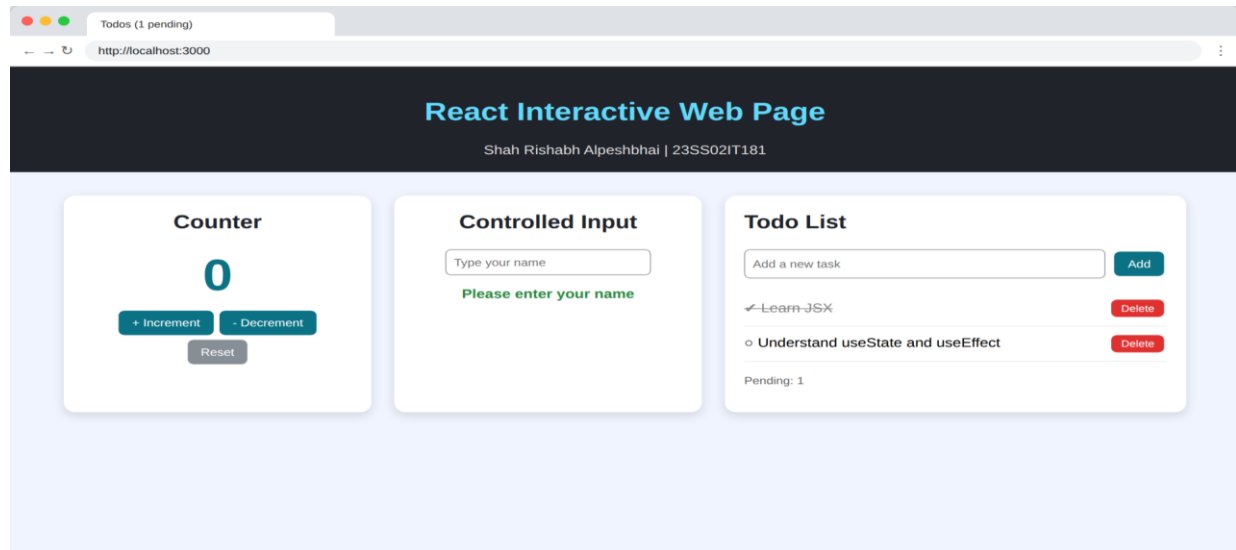
```

```
li span { cursor: pointer; }.done { text-decoration: line-through; color: #888; }  
.small { color: #666; font-size: 13px; }
```

### index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>React App</title>  
  <link rel="stylesheet" href="dist/index.css">  
</head>  
<body>  
  <div id="root"></div>  
  <script src="dist/index.js"></script>  
</body>  
</html>
```

### OUTPUT:



## PRACTICAL 6

To design a web application with a Node.js back end using Express: routing, middleware, static files and a REST API.

### STEPS:

1. Initialise the project and install Express.
2. Create server.js with middleware and routes, then start the server with node server.js.

### CODE:

#### Commands

```
npm init -y
npm install express
node server.js
```

#### server.js

```
const express = require("express");
const path = require("path");
const app = express();
const PORT = 3000;

// ----- Middleware -----
app.use(express.json()); // parse JSON body
app.use(express.static(path.join(__dirname, "public"))); // serve static files
app.use((req, res, next) => { // custom logger
  console.log(`${req.method} ${req.url}`);
  next();
});

// ----- In-memory data -----
let students = [
  { id: 1, name: "Rishabh Shah", course: "B.Sc. IT" },
  { id: 2, name: "Aarav Patel", course: "B.Sc. IT" },
];

// ----- Routes -----
app.get("/", (req, res) => {
  res.sendFile(path.join(__dirname, "public", "index.html"));
});

app.get("/about", (req, res) => {
  res.send("<h2>About: Node.js + Express backend by Rishabh Shah (23SS02IT181)</h2>");
});

app.get("/api/students", (req, res) => {
  res.json(students);
});

app.get("/api/students/:id", (req, res) => {
  const s = students.find((x) => x.id === parseInt(req.params.id));
  if (!s) return res.status(404).json({ message: "Student not found" });
  res.json(s);
});

app.post("/api/students", (req, res) => {
  const { name, course } = req.body;
  if (!name || !course)
    return res.status(400).json({ message: "name and course are required" });
  const student = { id: students.length + 1, name, course };
  students.push(student);
  res.status(201).json(student);
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

### public/index.html

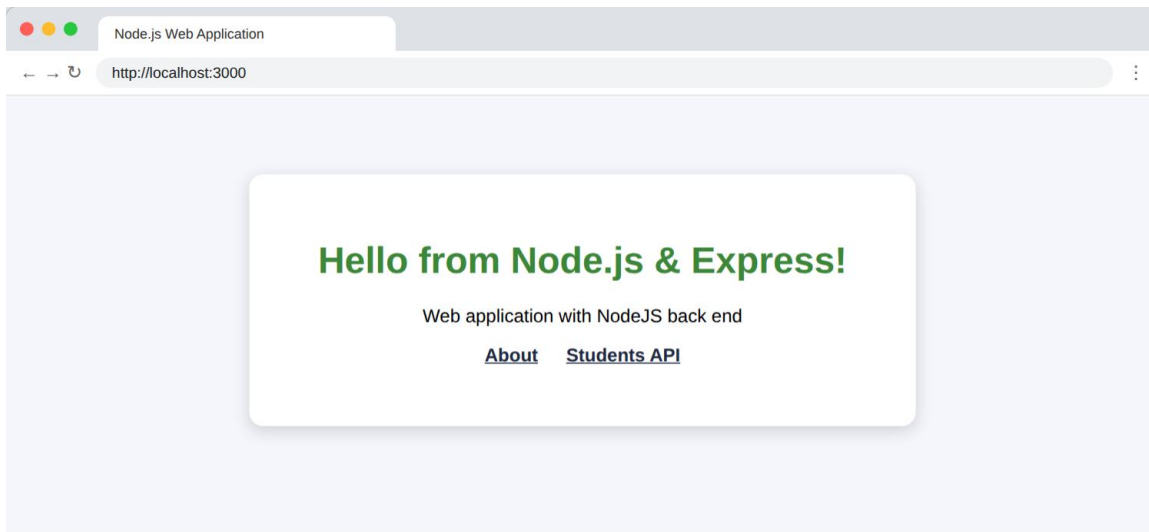
```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Node.js Web Application</title>
  <style>
    body { font-family: Arial; background:#f4f6fb; text-align:center; padding-top:60px; }
    .box { display:inline-block; background:#fff; padding:35px 60px; border-radius:12px; box-shadow:0 4px 14px rgba(0,0,0,.15); }
    h1 { color:#3c873a; } a { color:#1f2a44; font-weight:bold; margin:0 10px; }
  </style>
</head>
<body>
  <div class="box">
    <h1>Hello from Node.js & Express!</h1>
    <p>Web application with NodeJS back end</p>
    <p><a href="/about">About</a> <a href="/api/students">Students API</a></p>
  </div>
</body>
</html>
```

### OUTPUT:

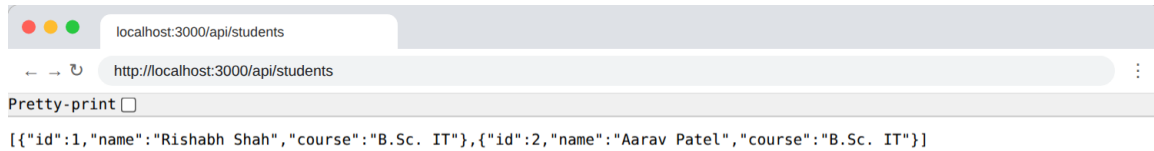
```
Windows PowerShell - node server.js

PS C:\Users\Rishabh\FSD\practical6> npm init -y
Wrote to C:\Users\Rishabh\FSD\practical6\package.json
PS C:\Users\Rishabh\FSD\practical6> npm install express
added 65 packages, and audited 66 packages in 3s

found 0 vulnerabilities
PS C:\Users\Rishabh\FSD\practical6> node server.js
Server running at http://localhost:3000
GET /api/students
GET /api/students
GET /api/students/2
POST /api/students
GET /api/students/9
```



*Output 2: Home page served from the public folder*



```
Windows PowerShell - API testing

PS C:\Users\Rishabh\FSD\practical6> curl.exe http://localhost:3000/api/students/2
{"id":2,"name":"Aarav Patel","course":"B.Sc. IT"}
PS C:\Users\Rishabh\FSD\practical6> curl.exe -X POST -H "Content-Type: application/json" -d '{"name":"Riya Mehta", "course":"B.Sc. IT"}' http://localhost:3000/api/students
{"id":3,"name":"Riya Mehta","course":"B.Sc. IT"}
PS C:\Users\Rishabh\FSD\practical6> curl.exe -i http://localhost:3000/api/students/9
HTTP/1.1 404 Not Found
{"message":"Student not found"}
```

## PRACTICAL 7

To connect Node.js with MongoDB and perform CRUD operations (Create, Read, Update, Delete) on a collection, and to integrate MySQL and execute SQL queries from Node.js.

### STEPS:

1. Install the official drivers: mongodb and mysql2.
2. Run mongo\_crud.js (MongoDB server on port 27017) and mysql\_crud.js (MySQL server on port 3306).

### CODE:

#### Commands

```
npm init -y
npm install mongodb mysql2
node mongo_crud.js
node mysql_crud.js
```

#### mongo\_crud.js

```
const { MongoClient } = require("mongodb");

const uri = "mongodb://127.0.0.1:27017";
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    console.log("Connected to MongoDB");
    const col = client.db("fsd_lab").collection("students");
    await col.deleteMany({});

    // ----- CREATE -----
    const ins = await col.insertMany([
      { name: "Rishabh Shah", enrollment: "23SS02IT181", sem: 7 },
      { name: "Aarav Patel", enrollment: "23SS02IT101", sem: 7 },
      { name: "Riya Mehta", enrollment: "23SS02IT145", sem: 7 },
    ]);
    console.log("\n[CREATE] Inserted documents:", ins.insertedCount);

    // ----- READ -----
    console.log("\n[READ] All students:");
    console.log(await col.find({}, { projection: { _id: 0 } }).toArray());

    // ----- UPDATE -----
    const upd = await col.updateOne(
      { enrollment: "23SS02IT101" },
      { $set: { sem: 8 } }
    );
    console.log("\n[UPDATE] Matched:", upd.matchedCount, "Modified:", upd.modifiedCount);
    console.log(await col.findOne({ enrollment: "23SS02IT101" }, { projection: { _id: 0 } }));

    // ----- DELETE -----
    const del = await col.deleteOne({ enrollment: "23SS02IT145" });
    console.log("\n[DELETE] Deleted:", del.deletedCount);

    console.log("\n[READ] After delete:");
    console.log(await col.find({}, { projection: { _id: 0 } }).toArray());
  } finally {
    await client.close();
  }
}
run().catch(console.error);
```

#### mysql\_crud.js

```
const mysql = require("mysql2/promise");

async function run() {
  // Connect to MySQL server
  const conn = await mysql.createConnection({
```

```

    host: "localhost",
    user: "root",
    password: "root",
  });
  console.log("Connected to MySQL");

  await conn.query("CREATE DATABASE IF NOT EXISTS fsd_lab");
  await conn.query("USE fsd_lab");
  await conn.query("DROP TABLE IF EXISTS students");
  await conn.query(`CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    enrollment VARCHAR(20) UNIQUE,
    sem INT
  )`);
  console.log("Table 'students' created");

  // INSERT
  await conn.query(
    "INSERT INTO students (name, enrollment, sem) VALUES ?",
    [
      ["Rishabh Shah", "23SS02IT181", 7],
      ["Aarav Patel", "23SS02IT101", 7],
      ["Riya Mehta", "23SS02IT145", 7],
    ]
  );
  console.log("3 rows inserted");

  // SELECT
  let [rows] = await conn.query("SELECT * FROM students");
  console.log("\nSELECT * FROM students");
  console.table(rows);

  // UPDATE
  const [u] = await conn.query("UPDATE students SET sem = 8 WHERE enrollment = ?", ["23SS02IT101"]);
  console.log("\nUPDATE -> affected rows:", u.affectedRows);

  // DELETE
  const [d] = await conn.query("DELETE FROM students WHERE enrollment = ?", ["23SS02IT145"]);
  console.log("DELETE -> affected rows:", d.affectedRows);

  // SELECT with condition
  [rows] = await conn.query("SELECT name, sem FROM students WHERE sem >= ? ORDER BY name", [7]);
  console.log("\nSELECT name, sem FROM students WHERE sem >= 7 ORDER BY name");
  console.table(rows);

  await conn.end();
}
run().catch(console.error);

```

## OUTPUT:

```

Windows PowerShell - MongoDB CRUD

PS C:\Users\Rishabh\FSD\practical7> npm install mongodb mysql2
added 24 packages, and audited 25 packages in 4s

found 0 vulnerabilities
PS C:\Users\Rishabh\FSD\practical7> node mongo_crud.js
Connected to MongoDB

[CREATE] Inserted documents: 3

[READ] All students:
[
  { name: 'Rishabh Shah', enrollment: '23SS02IT181', sem: 7 },
  { name: 'Aarav Patel', enrollment: '23SS02IT101', sem: 7 },
  { name: 'Riya Mehta', enrollment: '23SS02IT145', sem: 7 }
]

```

```
Windows PowerShell - MongoDB CRUD (contd.)

[UPDATE] Matched: 1 Modified: 1
{ name: 'Aarav Patel', enrollment: '23SS02IT101', sem: 8 }

[DELETE] Deleted: 1

[READ] After delete:
[
  { name: 'Rishabh Shah', enrollment: '23SS02IT181', sem: 7 },
  { name: 'Aarav Patel', enrollment: '23SS02IT101', sem: 8 }
]
```

```
Windows PowerShell - MySQL CRUD

PS C:\Users\Rishabh\FSD\practical7> node mysql_crud.js
Connected to MySQL
Table 'students' created
3 rows inserted

SELECT * FROM students



| (index) | id | name           | enrollment    | sem |
|---------|----|----------------|---------------|-----|
| 0       | 1  | 'Rishabh Shah' | '23SS02IT181' | 7   |
| 1       | 2  | 'Aarav Patel'  | '23SS02IT101' | 7   |
| 2       | 3  | 'Riya Mehta'   | '23SS02IT145' | 7   |



UPDATE -> affected rows: 1
DELETE -> affected rows: 1

SELECT name, sem FROM students WHERE sem >= 7 ORDER BY name



| (index) | name           | sem |
|---------|----------------|-----|
| 0       | 'Aarav Patel'  | 8   |
| 1       | 'Rishabh Shah' | 7   |


```

```
MySQL Command Line Client

PS C:\Users\Rishabh\FSD\practical7> mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
mysql> USE fsd_lab;
Database changed
mysql> SELECT * FROM students;
+-----+-----+-----+-----+
| id | name          | enrollment | sem |
+-----+-----+-----+-----+
| 1 | Rishabh Shah | 23SS02IT181 | 7 |
| 2 | Aarav Patel  | 23SS02IT101 | 8 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

## PRACTICAL 8

To develop a complete web application as a mini project: a Task Manager using HTML, CSS, JavaScript, Node.js, Express and MongoDB.

### STEPS:

1. Create the project, install express and mongodb.
2. Build the REST API in server.js (Create, Read, Update, Delete) and the front end in the public folder.
3. Start the server with node server.js and open <http://localhost:4000> in the browser.

### CODE:

#### Commands

```
npm init -y
npm install express mongodb
node server.js
```

#### server.js

```
const express = require("express");
const { MongoClient, ObjectId } = require("mongodb");
const path = require("path");

const app = express();
const PORT = 4000;
const client = new MongoClient("mongodb://127.0.0.1:27017");
let tasks;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

// READ - all tasks
app.get("/api/tasks", async (req, res) => {
  res.json(await tasks.find().sort({ _id: 1 }).toArray());
});

// CREATE - new task
app.post("/api/tasks", async (req, res) => {
  const { title, priority } = req.body;
  if (!title || !title.trim()) return res.status(400).json({ message: "Title is required" });
  const task = { title: title.trim(), priority: priority || "Medium", done: false };
  const r = await tasks.insertOne(task);
  res.status(201).json({ ...task, _id: r.insertedId });
});

// UPDATE - toggle done / edit
app.put("/api/tasks/:id", async (req, res) => {
  await tasks.updateOne({ _id: new ObjectId(req.params.id) }, { $set: req.body });
  res.json({ message: "Updated" });
});

// DELETE
app.delete("/api/tasks/:id", async (req, res) => {
  await tasks.deleteOne({ _id: new ObjectId(req.params.id) });
  res.json({ message: "Deleted" });
});

client.connect().then(async () => {
  tasks = client.db("taskmanager").collection("tasks");
  await tasks.deleteMany({});
  console.log("MongoDB connected");
  app.listen(PORT, () => console.log(`Task Manager running at http://localhost:${PORT}`));
});
```

#### public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Task Manager - Mini Project</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
  <h1>&#9989; Task Manager</h1>
  <p class="sub">Full Stack Mini Project &mdash; Node.js + Express + MongoDB</p>

  <div class="add">
    <input id="title" placeholder="Enter a new task...">
    <select id="priority">
      <option>High</option><option selected>Medium</option><option>Low</option>
    </select>
    <button onclick="addTask()">Add Task</button>
  </div>
  <p id="msg" class="msg"></p>

  <div class="stats">
    <span id="total">Total: 0</span>
    <span id="pending">Pending: 0</span>
    <span id="completed">Completed: 0</span>
  </div>

  <ul id="list"></ul>
</div>
<script src="app.js"></script>
</body>
</html>

```

### public/style.css

```

* { box-sizing: border-box; }
body { margin: 0; font-family: 'Segoe UI', Arial, sans-serif; background: linear-gradient(135deg,#667eea,#764ba2); min-height: 100vh; padding: 30px 12px; }
.container { max-width: 640px; margin: auto; background: #fff; border-radius: 14px; padding: 28px; box-shadow: 0 10px 30px rgba(0,0,0,.25); }
h1 { margin: 0; color: #3b3b98; } .sub { color: #777; margin-top: 4px; }
.add { display: flex; gap: 8px; margin-top: 18px; }
.add input { flex: 1; padding: 10px; border: 1px solid #bbb; border-radius: 8px; }
.add select { padding: 10px; border-radius: 8px; border: 1px solid #bbb; }
button { background: #3b3b98; color: #fff; border: 0; padding: 10px 16px; border-radius: 8px; cursor: pointer; }
button:hover { background: #575fcf; }
.msg { color: #d63031; min-height: 18px; margin: 6px 0; font-size: 14px; }
.stats { display: flex; justify-content: space-between; background: #f1f2f6; padding: 10px 14px; border-radius: 8px; font-weight: 600; font-size: 14px; }
ul { list-style: none; padding: 0; margin-top: 14px; }
li { display: flex; align-items: center; gap: 10px; padding: 12px; border-bottom: 1px solid #eee; }
li .t { flex: 1; cursor: pointer; } li.done .t { text-decoration: line-through; color: #999; }
.badge { font-size: 12px; padding: 3px 10px; border-radius: 20px; color: #fff; }
.High { background: #e74c3c; } .Medium { background: #f39c12; } .Low { background: #27ae60; }
.del { background: #e74c3c; padding: 6px 12px; font-size: 13px; }

```

### public/app.js

```

const API = "/api/tasks";

async function loadTasks() {
  const res = await fetch(API);
  const tasks = await res.json();
  const list = document.getElementById("list");
  list.innerHTML = "";

  tasks.forEach((t) => {
    const li = document.createElement("li");
    if (t.done) li.className = "done";
    li.innerHTML = `
      <input type="checkbox" ${t.done ? "checked" : ""} onchange="toggleTask('${t._id}', ${!t.done})">
      <span class="t" onclick="editTask('${t._id}', '${t.title.replace(/'/g, '\\')}')">${t.title}</span>
      <span class="badge ${t.priority}">${t.priority}</span>
      <button class="del" onclick="deleteTask('${t._id}')">Delete</button>`;
    list.appendChild(li);
  });

  const done = tasks.filter((t) => t.done).length;

```

```

document.getElementById("total").textContent = "Total: " + tasks.length;
document.getElementById("pending").textContent = "Pending: " + (tasks.length - done);
document.getElementById("completed").textContent = "Completed: " + done;
}

async function addTask() {
  const title = document.getElementById("title").value;
  const priority = document.getElementById("priority").value;
  const res = await fetch(API, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, priority }),
  });
  const msg = document.getElementById("msg");
  if (!res.ok) { msg.textContent = (await res.json()).message; return; }
  msg.textContent = "";
  document.getElementById("title").value = "";
  loadTasks();
}

async function toggleTask(id, done) {
  await fetch(`${API}/${id}`, { method: "PUT", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ done }) });
  loadTasks();
}

async function editTask(id, oldTitle) {
  const title = prompt("Edit task", oldTitle);
  if (title && title.trim()) {
    await fetch(`${API}/${id}`, { method: "PUT", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ title: title.trim() }) });
    loadTasks();
  }
}

async function deleteTask(id) {
  await fetch(`${API}/${id}`, { method: "DELETE" });
  loadTasks();
}

loadTasks();

```

## OUTPUT:

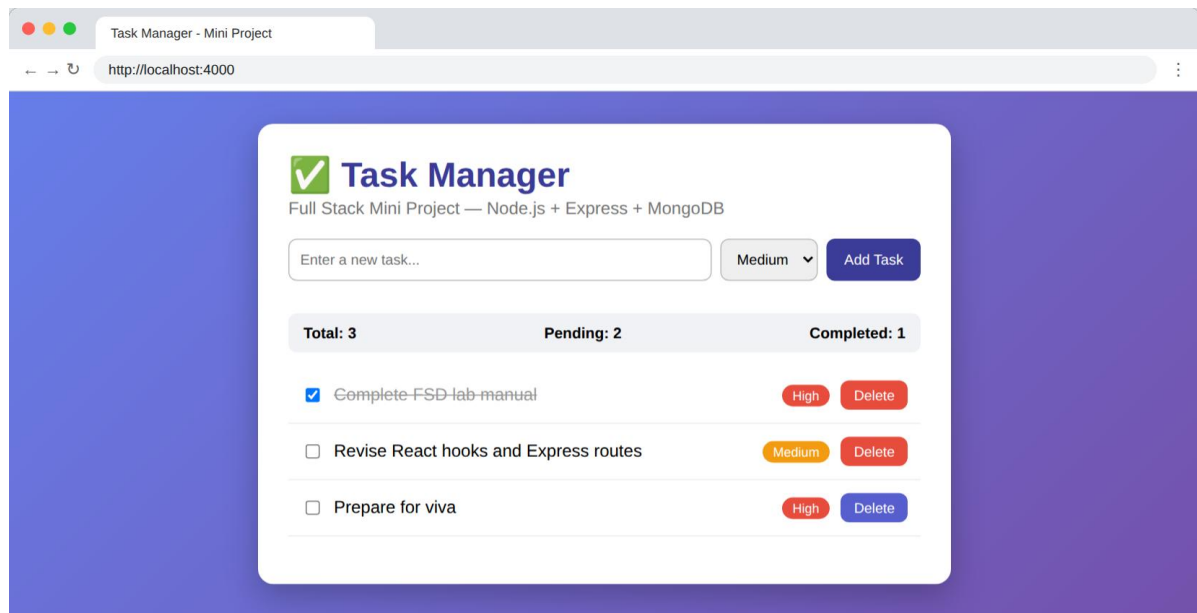
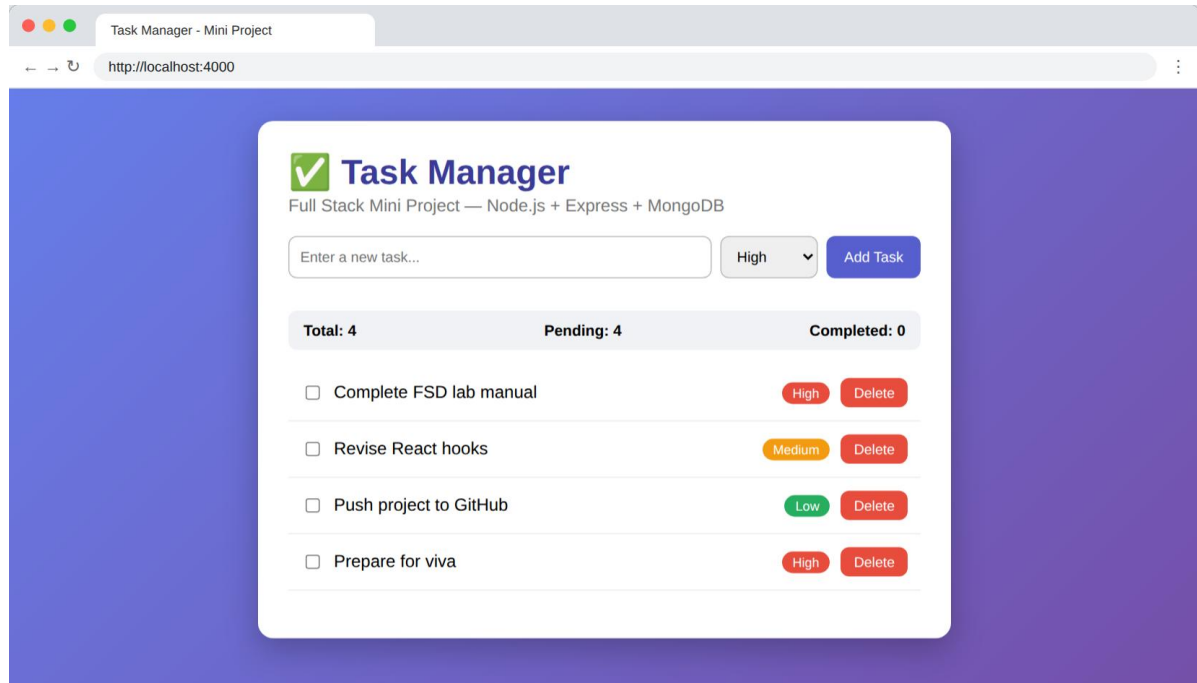
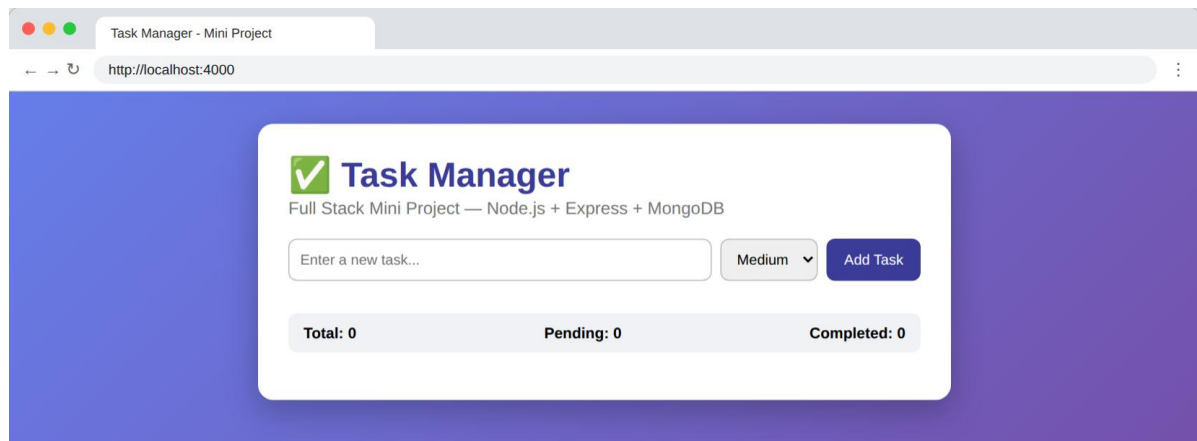
```

Windows PowerShell - node server.js

PS C:\Users\Rishabh\FSD\mini-project> npm init -y
PS C:\Users\Rishabh\FSD\mini-project> npm install express mongodb
added 70 packages, and audited 71 packages in 5s

found 0 vulnerabilities
PS C:\Users\Rishabh\FSD\mini-project> node server.js
MongoDB connected
Task Manager running at http://localhost:4000

```





**P P SAVANI**  
UNIVERSITY



NH 8, GETCO, Near Biltech,  
Village: Dhamdod, Kosamba,  
Ta.: Mangrol, Dist.: Surat - 394125.

Tel.: 02629 - 296788

Mobile: +91 9879608000

E-mail: [info@ppsu.ac.in](mailto:info@ppsu.ac.in)

[www.ppsu.ac.in](http://www.ppsu.ac.in)

